

# MILESTONE 4

## Building the Interface

### CliniScan AI — Chest X-Ray Analysis Platform

AI-Powered Web Interface with Deep Learning Backend

|            |                                                  |
|------------|--------------------------------------------------|
| Project    | CliniScan AI — Chest X-Ray Abnormality Detection |
| Milestone  | Milestone 4 — Building the Interface             |
| Tech Stack | FastAPI · EfficientNet-B3 · YOLOv8m · Grad-CAM   |
| Dataset    | VinDr-CXR (18,000 chest X-rays)                  |
| Model AUC  | 91.93% Mean AUC across 14 pathology classes      |
| Deployment | Hugging Face Spaces (Docker)                     |

## 1. Introduction

CliniScan AI is a web-based diagnostic assistance platform that uses deep learning to detect abnormalities in chest X-ray images. Milestone 4 covers the design and development of the complete user interface that connects the trained AI models to the end user.

The interface is built using FastAPI on the backend and plain HTML, CSS, and JavaScript on the frontend — all served from a single Python file (app.py). No external JavaScript frameworks are used. The application supports patient registration, X-ray upload, AI inference, result visualization, report generation, and analysis history tracking.

## 2. Pages and Navigation

The application is a Single Page Application (SPA) with four pages. Navigation happens instantly without page reloads. Each page is shown or hidden using a CSS active class controlled by JavaScript.

| Page          | What It Does                                                               |
|---------------|----------------------------------------------------------------------------|
| Overview      | Landing page — shows model info, tech stack, 14 class list, and AUC scores |
| Analyze X-Ray | Patient form, image upload, inference settings, and results display        |
| View Report   | Full on-screen clinical report after analysis                              |
| History       | All past analyses with search, filter, and CSV export                      |

## 3. Patient Details Module

---

A Patient Details form is shown at the top of the Analyze page. All three fields are mandatory. If any field is left empty and the user clicks Run Analysis, a red error message appears below that field and the form highlights it in red. The error clears as soon as the user starts typing.

| Field     | Type   | Required |
|-----------|--------|----------|
| Full Name | Text   | Yes      |
| Age       | Number | Yes      |
| Phone No. | Tel    | Yes      |

Patient information flows through the entire app — it appears in the results panel, on-screen report, history cards, history modal, downloaded PDF, and CSV export.

## 4. Image Upload and Settings

---

### 4.1 Image Upload

The upload zone supports both click-to-browse and drag-and-drop. When a file is selected, the FileReader API reads it and shows an instant preview inside the upload box. The Analyze button becomes active only after an image is uploaded.

### 4.2 Detection Settings

Before running analysis the user can adjust two detection parameters using sliders:

- Confidence Threshold (default 0.15) — minimum confidence for a YOLO detection to appear
- IoU Threshold (default 0.45) — overlap threshold for suppressing duplicate detections
- Grad-CAM Toggle — turn the heatmap on or off
- Bounding Box Toggle — turn YOLO detection on or off

## 5. AI Analysis Pipeline

---

When the user clicks Run AI Analysis, the frontend sends the image and patient details to the /analyze API endpoint. The backend runs the following steps in order:

- Image is decoded and resized to 384 × 384 pixels using OpenCV and Albumentations
- EfficientNet-B3 classifies the image and outputs probabilities for all 14 pathology classes
- YOLOv8m detects lesions and returns bounding boxes with class labels and confidence scores
- Grad-CAM generates a heatmap overlay showing which regions influenced the top prediction
- All results are base64-encoded and returned as a single JSON response to the browser

*Inference time is typically 2–5 seconds on GPU and 15–30 seconds on free CPU tier.*

## 6. Results Display

After analysis, results appear in a tabbed panel on the right side of the Analyze page. Three tabs organize the output:

| Tab          | Content                                                              |
|--------------|----------------------------------------------------------------------|
| Detection    | YOLOv8m annotated image with colored bounding boxes and class labels |
| Grad-CAM     | Heatmap overlay on the original image for the top predicted class    |
| All Findings | Horizontal probability bars for all 14 classes with POS / neg badges |

Above the tabs, three summary cards show: total detections, number of positive findings (probability  $\geq 50\%$ ), and inference time. The probability bars are color coded — teal for below 50%, amber for 50–70%, and red for above 70%.

## 7. Report Generation

### 7.1 On-Screen Report

The View Report page renders a full clinical report in HTML after analysis. It includes a teal header, patient information, executive summary with an alert box, detection and Grad-CAM images side by side, a classification results table for all 14 classes, a detection results table with bounding box coordinates, and a per-class AUC performance table.

### 7.2 PDF Download

Clicking Download PDF sends the image to the `/report/pdf` endpoint. The backend uses the `fpdf2` library to generate a multi-page PDF and streams it directly to the browser. The PDF includes all the same sections as the on-screen report plus patient details printed at the top. No file is saved on the server.

## 8. Analysis History

Every completed analysis is automatically saved to session memory and shown on the History page as a card grid. Each card shows the X-ray thumbnail, patient name, positive findings count, detection count, and inference time.

The History page provides:

- Filter chips — All, Positive Findings, No Findings, With Detections
- Live search — matches filename, patient name, phone number, or pathology class
- Detail modal — click any card to see full results including images and all class scores
- CSV Export — downloads a spreadsheet with all session analyses including patient details
- Clear All — removes all entries from the current session

## 9. Dark / Light Theme

A theme toggle button in the navigation bar switches between dark and light mode instantly. The entire color system is built on CSS custom properties (variables). Switching the data-theme attribute on the HTML element updates all 25+ colors at once with no page reload.

The selected theme is saved in localStorage so it is restored automatically on the next visit. On first visit, the app detects the operating system's color scheme preference and applies it automatically.

## 10. Backend API

The FastAPI backend exposes the following endpoints:

| Method | Endpoint    | Purpose                                                     |
|--------|-------------|-------------------------------------------------------------|
| GET    | /           | Serves the full HTML single-page application                |
| POST   | /analyze    | Runs classification, detection, and Grad-CAM — returns JSON |
| POST   | /report/pdf | Generates and streams a PDF clinical report                 |
| GET    | /health     | Returns model load status and device info                   |

## 11. Technology Stack

| Component        | Tool / Library            | Purpose                             |
|------------------|---------------------------|-------------------------------------|
| Web Framework    | FastAPI                   | Backend API and HTML serving        |
| Classifier       | EfficientNet-B3 (PyTorch) | 14-class chest X-ray classification |
| Object Detector  | YOLOv8m (Ultralytics)     | Lesion bounding box detection       |
| Explainability   | Grad-CAM (custom)         | Heatmap visualization               |
| Image Processing | OpenCV + Albumentations   | Preprocessing and annotation        |
| PDF Generation   | fpdf2                     | Clinical PDF report creation        |

|            |                         |                             |
|------------|-------------------------|-----------------------------|
| Frontend   | HTML / CSS / JavaScript | UI — no external framework  |
| Deployment | Docker + Hugging Face   | Containerized cloud hosting |

## 12. Deployment

The application is deployed on Hugging Face Spaces using Docker. The Dockerfile uses a PyTorch base image with CUDA support, installs system libraries needed by OpenCV, creates a non-root user as required by Hugging Face, installs Python dependencies, and starts the app using Uvicorn on port 7860.

Model weight files (best\_efficientnet\_b3.pth and best.pt) are uploaded using Git LFS since they exceed the 10 MB file size limit. The app includes a demo mode fallback — if model files are not found, the interface still works with placeholder outputs.

| File                     | Purpose                                               |
|--------------------------|-------------------------------------------------------|
| app.py                   | Complete application — backend + frontend in one file |
| Dockerfile               | Container build instructions for Hugging Face Spaces  |
| requirements.txt         | Python package dependencies                           |
| best_efficientnet_b3.pth | EfficientNet-B3 trained model weights (Git LFS)       |
| best.pt                  | YOLOv8m trained model weights (Git LFS)               |

## 13. Conclusion

Milestone 4 delivers a fully functional web interface for the CliniScan AI platform. The interface connects all deep learning components into a clean and accessible workflow — a user can upload an X-ray, enter patient details, run AI analysis, view the results, and download a PDF report in under a minute.

The application is live on Hugging Face Spaces and can be accessed from any browser without any installation. The working link is provided below.

# WORKING APPLICATION LINK

CliniScan AI — Deployed on Hugging Face Spaces

## Live Application URL

<https://huggingface.co/spaces/miruthula-s/CliniScanLungAbnormaldetection>

|          |                            |
|----------|----------------------------|
| Platform | Hugging Face Spaces        |
| SDK      | Docker                     |
| Port     | 7860                       |
| Access   | Public — no login required |